

BDS-SMC2 UAV Rescue System

Field Experiment Directive

Letsoalo Maile | GNSS & Satellite Communication Dissertation

June 2026

1. Overview

This directive covers every task from today until the dissertation experiments are complete. Follow the steps in order. Do not skip sections.

Item	Detail
Project	BeiDou-3 Short Message Communication for UAV Search & Rescue
Student	Letsoalo Maile
Hardware	ESP32 Dev Board + BDS-3 RDSS Module (RS232-TTL)
Portal	http://bdrd.hwasmart.com/ Login: RCSSTEAP_3058_SM_1 / 123456
Hardware days	5 days
Total timeline	2 weeks
Research gaps	Gap 1 (encoding), Gap 2 (latency), Gap 3 (environments), Gap 6 (telemetry)

2. Before Hardware (PC — Do This Now)

2.1 Install Missing Library

Step	Action	Command / Note
1	Open terminal in project folder	<code>cd C:\Users\OMEN\OneDrive\Desktop\BDS-SMC2</code>
2	Install scipy for statistical p-values	<code>pip install scipy</code>
3	Verify all scripts run	<code>python python/gap2_analysis.py --demo</code>

2.2 Scout Your 12 Field Locations

Walk to each location **before** hardware day. Take a photo and note the GPS coordinates on your phone. Confirm access.

Environment	Locations needed	What to look for
Open sky	OS-1, OS-2, OS-3	Wide open, no obstruction above
Light canopy	LC-1, LC-2, LC-3	Trees overhead, sky still visible
Urban canyon	UC-1, UC-2, UC-3	Tall buildings either side, narrow sky
Indoor	IN-1, IN-2, IN-3	Ground floor window / no window / upper floor

■ Do NOT arrive at a hardware day without pre-scouted locations. You will lose 2 hours and miss a session.

3. Hardware Day 1 — INDOOR Setup + Encoding Gaps

Location: Lab / indoors. Duration: Full day.

Step	Action	Command / Note
1	Wire ESP32 to BDS module	RXD→GPIO16, TXD→GPIO17, VCC→3.3V, GND→GND, LED→GPIO27
2	Open BDS portal in browser	http://bdrd.hwasmart.com/ → Login: RCSSTEAP_3058_SM_1 / 123456
3	Upload firmware MODE=0 (ASCII)	Arduino IDE → esp32_sender.ino → set MODE=0 → Upload
4	Proof of life: wait for first TX on portal	Portal must show incoming message. If not, debug wiring.
5	Run Gap 2 Session A — Morning	python python/serial_logger.py --port COM3 --session morning --weather clear
6	Wait for 34 transmissions then Ctrl+C	~6 minutes at 10s intervals
7	Switch to MODE=1 (Binary), upload, run 10 TX	Arduino IDE → MODE=1 → Upload → confirm 10 TX on portal
8	Switch to MODE=2 (Huffman), upload, run 10 TX	Arduino IDE → MODE=2 → Upload → confirm 10 TX on portal
9	Push all data to GitHub	git add data/ git commit -m 'day1: indoor + gap2 session A' git push

■ If proof of life fails: check RS232-TTL adapter orientation, re-seat all jumpers, confirm portal login before blaming firmware.

4. Hardware Day 2 — OUTDOOR Open Sky

Location: Pre-scouted open sky spots. Duration: Full day.

Step	Action	Command / Note
1	Go to OS-1, start logger	python python/field_test_logger.py --env open_sky --location OS-1 --gps "LAT,LON" --weather clear --n 20
2	Wait for 20 TX, Ctrl+C	~5 mins TX + setup. Note actual GPS from phone.
3	Move to OS-2, repeat	--location OS-2 (same command, change location)
4	Move to OS-3, repeat	--location OS-3
5	Run Gap 2 Session B — Midday	python python/serial_logger.py --port COM3 --session midday --weather clear
6	Wait for 33 TX then Ctrl+C	~6 minutes

7	Push data immediately — do not wait until end of day	git add data/ git commit -m 'day2: open sky OS-1/2/3 + gap2 midday' git push
---	--	--

5. Hardware Day 3 — OUTDOOR Light Canopy

Location: Pre-scouted canopy spots. Duration: Full day.

Step	Action	Command / Note
1	Go to LC-1, start logger	python python/field_test_logger.py --env light_canopy --location LC-1 --gps "LAT,LON" --obstruction 20 --n 20
2	Wait for 20 TX, Ctrl+C	Note sky obstruction estimate (0-100%).
3	Move to LC-2, repeat	--location LC-2
4	Move to LC-3, repeat	--location LC-3
5	Push data	git add data/ git commit -m 'day3: light canopy LC-1/2/3' git push

6. Hardware Day 4 — OUTDOOR Urban Canyon

Location: Pre-scouted urban canyon spots. Duration: Full day.

Step	Action	Command / Note
1	Go to UC-1, start logger	python python/field_test_logger.py --env urban_canyon --location UC-1 --gps "LAT,LON" --obstruction 60 --n 20
2	Wait for 20 TX, Ctrl+C	
3	Move to UC-2, repeat	--location UC-2
4	Move to UC-3, repeat	--location UC-3
5	Run Gap 2 Session C — Evening	python python/serial_logger.py --port COM3 --session evening --weather clear
6	Wait for 33 TX then Ctrl+C	~6 minutes
7	Push all data	git add data/ git commit -m 'day4: urban canyon UC-1/2/3 + gap2 evening' git push

7. Hardware Day 5 — INDOOR Environments

Location: Pre-scouted indoor spots. Duration: Half day.

Step	Action	Command / Note
1	Go to IN-1 (ground floor, near window)	python python/field_test_logger.py --env indoor --location IN-1 --obstruction 90 --n 20
2	Wait for 20 TX, Ctrl+C	
3	Move to IN-2 (ground floor, no window)	--location IN-2 --obstruction 100

4	Move to IN-3 (upper floor)	--location IN-3 --obstruction 80
5	Final push — all experiments complete	git add data/ git commit -m 'day5: all indoor complete' git push

■ After Day 5 all hardware work is done. Do not run more TX. Move to analysis immediately.

8. After All Hardware Days — Analysis (PC Only)

Run these three commands in order. Each prints results to the console and saves figures/CSV files automatically.

Step	Action	Command / Note
1	Run Gap 2 latency analysis (ANOVA + UAV error model)	python python/gap2_analysis.py
2	Run Gap 3 environmental analysis (chi-square + Fisher's exact + Bonferroni)	python python/gap3_analysis.py
3	Regenerate all 6 figures including Gap 3 bar chart	python generate_figures.py
4	Check figures/ folder — all PNG files are paper-ready at 150 DPI	figures/ folder in project root
5	Numbers from console output go directly into your paper tables	Copy mean, std, CI, p-values into Lab7_Report.md

9. Quick Command Reference

Script	When	Command
serial_logger	Gap 2 TX session	python python/serial_logger.py --port COM3 --session morning --weather clear
field_test_logger	Gap 3 field TX	python python/field_test_logger.py --env open_sky --location OS-1 --gps "30.4196,120.2977" --n 20
gap2_analysis	After hardware	python python/gap2_analysis.py
gap3_analysis	After hardware	python python/gap3_analysis.py
generate_figures	After analysis	python generate_figures.py
git push	After every location	git add data/ && git commit -m 'message' && git push
field summary	Check progress	python python/field_test_logger.py --summary

10. Risk Mitigation

Risk	If it happens	Recovery
Hardware doesn't work Day 1	Give full Day 1 to debugging	Reduce Gap 3 to 20 TX x 4 environments (original plan). Still publishable.
Rain on outdoor day	Reschedule that environment to next available day	Indoor environments can fill a rain day

Portal login fails	Log in the night before each hardware day	Use User 2: RCSSTEAP_3058_SM_2 / 123456
Laptop crashes mid-session	Push CSV after every location — not end of day	Data already on GitHub. Re-run only that location.
Location inaccessible	Scout day before. Have a backup spot within 5 mins walk	GPS within 200m of original is acceptable